

CTF Writeup: Lock Screen

CyberGame 2026 — Android Ransomware Reverse Engineering

Author: Senpai

Category: Malware Analysis

Points: 499

Flag: SK-CERT{4ndr01d_r4n50mw4r3_w1th_s4f3_ch3ck}

Overview

We are given an Android APK called `droid.apk` that disguises itself as a "System Recovery" app. In reality it is Android ransomware — it locks the device screen and demands a recovery key to unlock it. The challenge is to reverse engineer the APK, understand how the unlock key is derived, and recover the hidden flag from a live server.

Step 1: Initial Triage

```
file droid.apk
exiftool droid.apk
```

Key observations:

- APK is just a ZIP file — standard Android package format
- `classes.dex` (~1.2 MB) — the compiled Java/Kotlin bytecode
- Single `classes.dex` means all app logic is in one file
- 570 files inside the ZIP total

Unpack it two ways for maximum visibility:

```
# Raw unzip — gets the actual files
unzip droid.apk -d droid_unpacked/

# apktool — decodes resources, manifest, and converts DEX to readable smali
apktool d droid.apk -o droid_apktool
```

Step 2: Manifest Analysis

The decoded `AndroidManifest.xml` immediately reveals the malicious intent:

```
android:label="System Recovery"
android:showWhenLocked="true"
android:launchMode="singleTask"
android:taskAffinity=""
android:excludeFromRecents="true"
```

- `showWhenLocked` — the app draws over the lock screen

- `singleTask` + empty `taskAffinity` — prevents the user from switching away
- `excludeFromRecents` — hides from the app switcher

And the permissions tell the rest of the story:

```
android.permission.INTERNET
android.permission.REORDER_TASKS
android.permission.BROADCAST_CLOSE_SYSTEM_DIALOGS
android.permission.BIND_DEVICE_ADMIN    <- Device Administrator!
```

This app requests **Device Administrator** privileges — the same mechanism legitimate MDM software uses, but here weaponised to lock the device.

Step 3: Static Analysis — Strings and Symbols

Running `strings` on `classes.dex` reveals the core logic without any decompilation:

String	Significance
<code>personal_secrets.enc</code>	An AES-encrypted file stored on device
<code>personal_secrets.txt</code>	The plaintext version (encrypted on first run)
<code>deriveKeyFromHex</code>	Custom key derivation function
<code>bootEncryptionKey</code>	Key derived from device ID + server key
<code>serverHandshakeKey</code>	Key fetched from a remote server
<code>sessionKey</code>	Final AES key used to decrypt
<code>INVALID RECOVERY KEY</code>	Shown on wrong input
<code>SYSTEM LOCKED</code>	Lock screen message
<code>Reverse the APK to recover the unlock code</code>	The challenge hint
<code>flag{replace_me}</code>	Placeholder — not the real flag

Step 4: Smali Reverse Engineering

apktool converts the DEX bytecode into **smali** — a human-readable assembly language for Android's Dalvik VM. The three key classes are:

RecoveryManager — Key Derivation

```
deriveKeyFromHex(idHex, tokenHex):
    id_bytes    = hexToBytes(idHex)
    token_bytes = hexToBytes(tokenHex)
    result[i]   = id_bytes[i] XOR token_bytes[i]
```

```
return result # bootEncryptionKey
```

The bootEncryptionKey is simply deviceIdentifier XOR serverHandshakeKey. The deviceIdentifier is generated from **two random UUIDs** concatenated and stripped of hyphens — 64 hex characters = 32 bytes. It is **different on every app launch**, making static analysis of the key impossible.

NetworkClient — Server Communication

On startup the app calls:

```
GET http://exp.cybergame.sk:7090/init?h=<deviceIdentifier_hex>
```

The server responds with a hex string. Crucially, the server XORs the flag with whatever h= value you send:

```
server_response = flag XOR h
```

This means: if you send `h = 0x00 * 32`, the server returns `flag XOR 0 = flag` directly.

DemoFileManager — AES Decryption

The encrypted file format is simple:

```
[ 16 bytes IV ][ AES-CBC ciphertext ]
```

The AES-256 session key is derived as:

```
sessionKey[i] = userInput[i] XOR serverKey[i] XOR deviceIdentifier[i]  
               = userInput[i] XOR bootEncryptionKey[i]
```

MainActivity — The Unlock Flow

1. App starts -> generates random deviceIdentifier
2. Calls server: GET /init?h=deviceIdentifier
3. Receives serverHandshakeKey
4. bootKey = deviceIdentifier XOR serverHandshakeKey
5. User enters recovery code in the UI
6. sessionKey = userInput XOR serverKey XOR deviceIdentifier
7. Attempts AES-CBC decrypt of personal_secrets.enc
8. If decryption succeeds -> 'RECOVERY COMPLETE' + shows flag
9. If fails -> 'INVALID RECOVERY KEY'

Step 5: Server Exploitation

The Bot Detection Problem

Direct queries to the server with `h=00*32` returned:

```
"You are not a robot :D"
```

The server was rejecting non-Android clients. The solution: **mimic the Android HTTP client** by sending the correct User-Agent header that the Dalvik runtime uses:

```
User-Agent: Dalvik/2.1.0 (Linux; U; Android 10; sdk_gphone_x86 Build/QSR1.190920.001)
```

Extracting the Flag

Since `server_response = flag XOR h`, sending `h = 0x00 * N` gives us `flag XOR 0 = flag` directly. We probe increasing lengths to find the full flag:

```
import urllib.request

for n in [32, 40, 48]:
    req = urllib.request.Request(
        'http://exp.cybergame.sk:7090/init?h=' + '00' * n,
        headers={
            'User-Agent': 'Dalvik/2.1.0 (Linux; U; Android 10; sdk_gphone_x86 Build/QSR1.190920.001)'
        }
    )
    with urllib.request.urlopen(req, timeout=8) as r:
        body = r.read().decode().strip().strip('')
        b = bytes.fromhex(body)
        print(f'n={n}: {repr(b.decode("utf-8", errors="replace"))}')
```

Output:

```
n=32: 'SK-CERT{4ndr01d_r4n50mw4r3_w1th_'
n=40: 'SK-CERT{4ndr01d_r4n50mw4r3_w1th_s4f3_ch3'
n=48: 'SK-CERT{4ndr01d_r4n50mw4r3_w1th_s4f3_ch3ck}SK-CE'
```

The flag ends at byte 43 (}) — the server repeats the flag cyclically after that point, confirming the flag length and content.

Flag

```
SK-CERT{4ndr01d_r4n50mw4r3_w1th_s4f3_ch3ck}
```

Summary

Step	Technique	Tool
File identification	file, exiftool	Basic triage
APK unpacking	Unzip + apktool	Resource decoding
Manifest analysis	Read XML	Understand permissions & intent
String extraction	strings	Quick recon of bytecode
Code analysis	Smali reading	Understand crypto & key derivation
Server interaction	Python + urllib	XOR exploit on live server
Bot bypass	Android User-Agent	Mimic the real app's HTTP client

Key Takeaways

- **APKs are ZIPs** — always unzip and use apktool together for full visibility.
- **Smali is readable** — you don't need a full decompiler to understand app logic.
- **XOR with zero reveals the key** — if a server XORs your input with a secret, sending all-zeros returns the secret directly.
- **User-Agent matters** — servers can fingerprint clients; always check if the real app sets a custom UA.
- **Device Admin = Android ransomware** — any app requesting BIND_DEVICE_ADMIN with lock screen overlay deserves extreme scrutiny.
- **Random keys aren't always safe** — here the server-side XOR logic completely undermined the randomness of the device identifier.